

NanoLink API 参考手册

轻量级 TCP/IP 协议栈 — 从以太网帧到 TCP 连接的全栈 C 语言实现
每个函数配有来自项目实际场景的详细示例，展示完整的调用上下文和错误处理。

1. osal — 操作系统适配层

提供跨平台的内存分配、时间获取等基础服务。整个协议栈只有这个文件感知操作系统。移植到 FreeRTOS、裸机等平台时，只需替换此文件。

osal_init

```
void osal_init(void);
```

描述：初始化操作系统适配层。Linux 下为空操作，FreeRTOS 等平台在此初始化堆和时钟。

示例：

```
// main() 中最先调用。虽然 Linux 下是空函数，但保留这个调用保证了移植性——  
// 移植到 FreeRTOS 时，只需要在 osal_init 里初始化内存池和时钟，  
// 所有上层代码不用改。  
osal_init();
```

osal_malloc

```
void *osal_malloc(size_t size);
```

描述：分配指定大小的内存块。Linux 下直接调用 malloc，裸机下可用静态池实现。

返回值：成功返回内存指针，失败返回 NULL

示例：

```
// 场景：UDP 模块需动态创建一个协议控制块（PCB）。
// 为什么不用 malloc? 如果将来要移植到 FreeRTOS (用 pvPortMalloc),
// 所有调用点都不用改，只要改这一个函数。
struct udp_pcb *udp_new(void)
{
    struct udp_pcb *pcb;
    pcb = (struct udp_pcb *)osal_malloc(sizeof(struct udp_pcb));
    if (pcb == NULL) {
        // 内存不够——在嵌入式设备上很常见，上层必须处理这个错误
        return NULL;
    }
    memset(pcb, 0, sizeof(struct udp_pcb));
    return pcb;
}
```

osal_free

```
void osal_free(void *ptr);
```

描述：释放 `osal_malloc` 分配的内存。ptr 为 NULL 时无事发生。

示例：

```
// UDP 控制块不再使用，归还内存。
// 注意 osal_free 之后不要再访问 pcb 的任何字段。
void udp_remove(struct udp_pcb *pcb)
{
    if (pcb == NULL) return;
    // 先从全局链表摘除...
    osal_free(pcb); // pcb 指针从此失效
}
```

osal_get_time_ms

```
uint32_t osal_get_time_ms(void);
```

描述：获取系统启动以来的单调时钟，单位毫秒。使用 `CLOCK_MONOTONIC`，不受系统时间调整影响。用于定时器驱动和超时计算。

返回值：毫秒级时间戳（从系统启动算起，约 49 天后回绕）

示例：

```
// 主循环中驱动 1 秒和 200ms 的定时任务。
// 不用 sleep—通过比较时间戳，只在到达间隔时才执行。
void main_loop_tick(void)
{
    static uint32_t last_tick = 0;    // 静态变量，跨调用保持
    static uint32_t last_fast = 0;
    uint32_t now = osal_get_time_ms();

    if (now - last_tick >= 1000) {    // 距上次慢定时已过 1 秒
        last_tick = now;
        timer_tick(1000);            // 驱动所有定时器
        arp_tick();                  // 检查 ARP 超时
    }
    if (now - last_fast >= 200) {    // 快定时 200ms
        last_fast = now;
        tcp_fast_tick();
    }
}
```

2. mbuf — 缓冲区管理

协议栈的数据载体，零拷贝设计的核心。数据包在各层之间传递时只移动 `data` 指针，不拷贝数据本身。

静态池：256 个 mbuf 结构体，启动时串成空闲链表
SMALL 类型：128 字节内嵌缓冲，其中 64 字节 headroom
LARGE 类型：外部动态缓冲，用于超长帧
引用计数：多处持有时不复制，仅当引用归零才回收

mbuf_init

```
void mbuf_init(void);
```

描述：把 256 个 mbuf 结构体用 next 指针串成单向空闲链表。协议栈启动时调用一次。

示例：

```
// main() 中, 在创建任何 mbuf 之前必须调用:
int main(void)
{
    osal_init();
    mbuf_init(); // 必须在 timer_init / netif_init 等之前
    timer_init();
    // 现在可以安全地 mbuf_alloc 了
}
```

mbuf_alloc

```
struct mbuf *mbuf_alloc(void);
```

描述: 从池中取一个 SMALL 类型 mbuf。data 指针指向 headroom 之后——即 buffer 起始 + 64 字节处。64 字节 headroom 足以容纳 ETH(14) + IP(20) + TCP(20) = 54 字节协议头。

返回值: 成功返回 mbuf 指针, 256 个全部在外面时返回 NULL

示例:

```
// ARP 请求包只有 28 字节, SMALL mbuf 足够了。
struct mbuf *m = mbuf_alloc();
if (m == NULL) {
    printf("mbuf 池耗尽! 256 个全在外面\n");
    return -1;
}
// m->len = 0, m->total_len = 0, m->type = SMALL
// m->data 指向 headroom 之后

// 标记我们要用 28 字节
mbuf_append(m, sizeof(struct arp_packet));
// 现在可以往 m->data[0..27] 填 ARP 包内容了
```

mbuf_alloc_large

```
struct mbuf *mbuf_alloc_large(uint16_t size);
```

描述: 分配一个 LARGE 类型 mbuf。外部缓冲用 osal_malloc 分配。用于超过 64 字节的帧 (如 1500 字节以太网帧)。LARGE mbuf 的 headroom 为 0。

返回值: 成功返回 mbuf 指针, 失败返回 NULL

示例:

```
// 收到一个 1500 字节的以太网帧—SMALL 的 64 字节数据区装不下。
void tap_poll(struct netif *ni)
{
    uint8_t raw[2048];
    ssize_t n = read(fd, raw, sizeof(raw));
    if (n <= 0) return;

    struct mbuf *m;
    if (n > 64) {
        // 大帧→LARGE
        m = mbuf_alloc_large((uint16_t)n);
        if (m == NULL) return; // osal_malloc 失败或池耗尽
    } else {
        // 小帧 (如 ARP 42B) →SMALL
        m = mbuf_alloc();
    }

    if (m == NULL || mbuf_append(m, (uint16_t)n) != 0) {
        mbuf_free(m); // append 失败后需要释放
        return;
    }

    mbuf_copy_from(m, raw, (uint16_t)n); // 把 TAP 读到的字节拷进去
    netif_input(ni, m); // 交给协议栈处理
}
```

mbuf_free

```
void mbuf_free(struct mbuf *m);
```

描述：引用计数减 1。归零后：

- 递归释放整条 next 链
- LARGE 类型释放外部缓冲 (osal_free)
- 把 mbuf 结构体归还 free_list

示例：

```
// 场景：ARP 挂起队列
// IP 层有一个包要发给尚未解析 MAC 的目标。
// arp_query 把 m 挂进队列前，增加引用计数：
mbuf_ref_inc(m);                // ref_count: 1 → 2
arp_pending_queue = m;

// IP 层不关心了，调 mbuf_free:
mbuf_free(m);                   // ref_count: 2 → 1, 不回收！

// ...稍后 ARP 解析完成，取出队列：
struct mbuf *m = dequeue(&pending_queue);
// 发送完毕：
mbuf_free(m);                   // ref_count: 1 → 0, 回收！
```

mbuf_prepend

```
int mbuf_prepend(struct mbuf *m, uint16_t len);
```

描述：在数据前面预留 len 字节——纯指针操作，零拷贝。data 指针前移 len 字节，len 和 total_len 各加 len。调用者随后往新预留的空间写入协议头。

返回值：0 成功，-1 headroom 不够

示例：

```
// 场景：UDP 模块构造完数据报 [UDP头|数据]，IP 层要在前面加 20 字节 IP 头。
// m 当前状态：
//   [--- headroom 64 ---][UDP头 8B|数据 100B]
//   ↑ buf_start          ↑ data (指向 UDP 头)

int ip_output(uint32_t dst_addr, uint8_t proto, struct mbuf *m)
{
    // 在 UDP 前面预留 20 字节给 IP 头
    if (mbuf_prepend(m, IP_HEADER_MIN_LEN) != 0) {
        // headroom 只有 64 字节，前面如果已经加了帧头(14)+IP头(20)就只剩 30
        // 不够就失败
        mbuf_free(m);
        return -1;
    }
    // 现在 m 布局：
    //   [--- headroom 44 ---][IP头 20B][UDP头 8B|数据 100B]
    //   ↑ buf_start          ↑ data (前移了 20)

    struct ip_header *ip = (struct ip_header *)m->data;
    ip->ver_ihl = 0x45;    // 版本 4，头长 5×4=20 字节
    ip->proto = proto;     // IPPROTO_UDP
    // ... 填其他字段
}
```

mbuf_append

```
int mbuf_append(struct mbuf *m, uint16_t len);
```

描述：在数据尾部扩展 len 字节。零拷贝，只修改 len 和 total_len。调用者随后自己往新增区域写数据。

返回值：0 成功，-1 容量不够

示例：

```
// 场景：应用层有 200 字节要发送，先 append 再 copy。
// 必须先 append 再 copy，因为 mbuf_copy_from 依赖 m->len 来判断可写空间。
int socket_send(struct socket *sock, const void *data, uint16_t len)
{
    struct mbuf *m = mbuf_alloc();
    if (m == NULL) return -1;

    // 先扩展长度
    if (mbuf_append(m, len) != 0) {
        // SMALL 只有 64 字节数据区，200 > 64 会失败
        // 应该用 mbuf_alloc_large 才对，这里演示错误处理
        mbuf_free(m);
        return -1;
    }
    // 再拷贝数据
    mbuf_copy_from(m, data, len);
    // 现在 m->data[0..199] 存着应用数据
}
```

mbuf_pull

```
int mbuf_pull(struct mbuf *m, uint16_t len);
```

描述：从数据头部剥离 len 字节——纯指针操作，零拷贝。data 后移 len 字节，len 和 total_len 各减 len。上层直接看到剥掉协议头之后的数据。

返回值：0 成功，-1 数据不够 len 字节

示例：


```

// 收包路径：逐层剥协议头
// 链路层收到完整帧：[ETH 14B|IP 20B|TCP 20B|数据]
//                               ↑ data
// 链路层处理完后：
ll_dispatch_input(ni, m) {
    // ... 根据以太网帧类型分发 ...
    mbuf_pull(m, 14); // 剥 ETH 头
    // 现在：[ETH 14B|IP 20B|TCP 20B|数据]
    //                               ↑ data — IP 层直接看到 IP 头！
    return ip_input(ni, m);
}

// IP 层处理完后：
ip_input(ni, m) {
    // ... 校验、查路由 ...
    mbuf_pull(m, 20); // 剥 IP 头
    // 现在：[ETH|IP|TCP 20B|数据]
    //                               ↑ data — TCP 层直接看到 TCP 头！
    return tcp_input(ni, m, src_ip, dst_ip);
}

// 整个过程一字节数据都没拷贝，只是移动了指针。

```

mbuf_ref_inc

```
void mbuf_ref_inc(struct mbuf *m);
```

描述：引用计数 +1。多个模块需要同时持有一个 mbuf 时使用——ARP 挂起是最典型的场景。

示例：

```
// 场景：一个包要发给 10.0.0.2，但该 IP 的 MAC 还不知道。
// IP 层调 arp_query，ARP 模块把包挂起，等解析完再发。
int arp_query(struct netif *ni, uint32_t ip_addr, uint8_t *mac_out,
               struct mbuf *m)
{
    // 表里没有这个 IP
    struct arp_entry *entry = &arp_table[free_slot];
    entry->state = PENDING;

    if (m != NULL) {
        mbuf_ref_inc(m);           // ref_count 1→2
        entry->pending_queue = m;   // ARP 持有引用
    }

    // IP 层返回后会调 mbuf_free(m)，ref_count 2→1，不回收
    send_arp_request(ni, ip_addr);
    return -1;
}

// ... ARP 回复到达，flush 挂起队列：
struct mbuf *pending = entry->pending_queue;
ni->send(ni, pending);
mbuf_free(pending); // ref_count 1→0，真正回收
```

mbuf_chain

```
void mbuf_chain(struct mbuf *m1, struct mbuf *m2);
```

描述：把 m2 链拼接到 m1 的 next 链尾部。m1 的 total_len 累加 m2 的 total_len。

示例：

```
// IP 分片重组：两个分片先后到达
struct mbuf *frag1 = ...; // [IP头|分片1数据 200B]
struct mbuf *frag2 = ...; // [分片2数据 150B]

// 拼接后成为一个完整 IP 包：
mbuf_chain(frag1, frag2);
// frag1: [IP头|分片1 200B] →next→ [分片2 150B]
// frag1->total_len = 350
// 调用 mbuf_copy_to(frag1, buf, 350) 就能拿到全部数据
```

mbuf_split

```
struct mbuf *mbuf_split(struct mbuf *m, uint16_t len);
```

描述：从 mbuf 链头部切下前 len 字节，返回剩余部分的新链头。切在片段边界时纯指针操作（零拷贝），切在片段内部时才拷贝。

返回值：剩余部分的链头，len 超出总长度返回 NULL

示例：

```
// 收到完整 IP 包后，IP 层把负载切出来交给 TCP 层
// m: [IP头 20B|TCP头 20B|数据 100B], total_len = 140
struct mbuf *payload = mbuf_split(m, 20); // 切掉 20 字节 IP 头
if (payload == NULL) {
    mbuf_free(m);
    return -1;
}
// 结果：
// m:      [IP头 20B]      total_len = 20
// payload: [TCP头 20B|数据 100B] total_len = 120
// 接下来 TCP 层处理 payload, IP 头可以释放
mbuf_free(m);
return tcp_input(ni, payload, src_ip, dst_ip);
```

mbuf_copy_to / mbuf_copy_from

```
uint16_t mbuf_copy_to(struct mbuf *m, void *buf, uint16_t len);
int mbuf_copy_from(struct mbuf *m, const void *buf, uint16_t len);
```

描述：

- **copy_to**：把 mbuf 链的数据**拷到**连续内存 buf。即使数据分布在多个 next 片段中，调用者也能拿到连续数据。
- **copy_from**：把连续内存 buf 的数据**拷到** mbuf 链。调用者无需关心 mbuf 内部的片段分布。

返回值： **copy_to** 返回实际拷贝字节数； **copy_from** 返回 0 成功、-1 空间不够

示例：

```

// socket_recv: 协议栈内部数据可能是 next 链拼起来的,
// 应用层只想要连续的 buf, copy_to 帮你处理这个差异。
char app_buf[1024];
int n = mbuf_copy_to(sock->recv_queue, app_buf, sizeof(app_buf));
// app_buf[0..n-1] 现在是连续数据, 应用层能直接使用

// ---

// socket_send: 应用层给的是连续 buf, copy_from 填进 mbuf 链。
// 注意: 必须先 mbuf_append 设置长度, 再 mbuf_copy_from 拷贝数据!
struct mbuf *m = mbuf_alloc();
mbuf_append(m, 100);           // 先声明 100 字节空间
mbuf_copy_from(m, app_data, 100); // 再拷贝
// 如果顺序反了 (先 copy 再 append), mbuf_copy_from 会因为 m->len=0
// 找不到可写空间而返回 -1。

```

mbuf_enqueue / mbuf_dequeue / mbuf_remove

```

void mbuf_enqueue(struct mbuf_list *q, struct mbuf *m);
struct mbuf *mbuf_dequeue(struct mbuf_list *q);
void mbuf_remove(struct mbuf_list *q, struct mbuf *m);

```

描述: mbuf 队列操作。操作 `next_pkt` 指针 (不是片段链的 `next`)。 `enqueue` 尾部入队; `dequeue` 头部出队; `remove` 可从任意位置摘除 ($O(n)$ 遍历)。

示例:

```
// ARP 挂起队列的完整生命周期
struct mbuf_list pending_q = {NULL, NULL, 0}; // head, tail, count

// 包 1 和包 2 都在等 ARP 解析
mbuf_enqueue(&pending_q, pkt1); // 队列: pkt1, count=1
mbuf_enqueue(&pending_q, pkt2); // 队列: pkt1→pkt2, count=2

// ARP 解析完成, 逐个取出发送
struct mbuf *p = mbuf_dequeue(&pending_q); // 取出 pkt1, count=1
// ... 填上 MAC, 发送 ...
mbuf_free(p);

p = mbuf_dequeue(&pending_q); // 取出 pkt2, count=0
// ... 发送 ...

// TCP 重传队列场景: 累积 ACK 确认了 pkt1~pkt3, 需要从队列移除
// 用 remove 可以定向删除, 不需要出队再入队
mbuf_remove(&unacked_q, pkt1); mbuf_free(pkt1);
mbuf_remove(&unacked_q, pkt2); mbuf_free(pkt2);
mbuf_remove(&unacked_q, pkt3); mbuf_free(pkt3);
```

mbuf_chain_len

```
uint16_t mbuf_chain_len(struct mbuf *m);
```

描述: 遍历 next 链, 累加每个片段的 len。通常直接读 `m->total_len` 即可——此函数仅在 `total_len` 可能不准确时使用 (如调试)。

示例:

```
// 调试: 验证 total_len 和实际片段总和是否一致
uint16_t actual = mbuf_chain_len(m);
if (actual != m->total_len) {
    printf("BUG: total_len=%d 但实际链长=%d\n", m->total_len, actual);
}
```

3. timer — 定时器服务

单次和周期定时器。TCP 重传、ARP 老化、IP 分片超时等均依赖此模块。外部周期性调用 `timer_tick()` 驱动, 不绑定特定时钟源。

架构：静态池（32 个）+ 双链表
free_list → 空闲定时器
active_list → 正在倒计时的定时器

timer_init

```
void timer_init(void);
```

描述：把 timer_pool[32] 用 next 指针串成空闲链表。

示例：

```
int main(void)
{
    mbuf_init();
    timer_init(); // 必须在任何 timer_create 之前调用
    // 现在 32 个定时器都在 free_list 上待命
}
```

timer_create

```
timer_handle_t timer_create(uint32_t interval, uint8_t type,
                             timer_callback_fn callback, void *arg);
```

描述：从池中分配一个定时器，填充参数。此时尚未启动——需要调 timer_start 才开始倒计时。

参数：

- interval — 定时周期（毫秒）
- type — TIMER_TYPE_ONCE (0) 到期一次后停止；TIMER_TYPE_PERIODIC (1) 到期后自动重装
- callback — 到期回调函数，签名为 void (*)(void *arg)
- arg — 传给回调的参数，通常是 PCB 指针

返回值：成功返回句柄，32 个全部分配出去时返回 NULL

示例：

```
// TCP 创建超时重传定时器: 500ms 内没收到 ACK, 触发重传
// 这个回调稍后会在 timer_tick 中被调用:
void tcp_retransmit_callback(void *arg)
{
    struct tcp_pcb *pcb = (struct tcp_pcb *)arg;
    printf("重传! 这是第 %d 次\n", ++pcb->retransmit_count);
    // 重新发送未确认的数据...
}

struct tcp_pcb *pcb = tcp_new();
pcb->retransmit_timer = timer_create(
    500,                                // 500ms 超时
    TIMER_TYPE_ONCE,                    // 单次—重传后需要手动 restart
    tcp_retransmit_callback,           // 到期回调
    pcb                                 // 把 PCB 指针传进去
);
if (pcb->retransmit_timer == NULL) {
    printf("32 个定时器全部用完了! \n");
}
// 此时定时器还没跑, 需要 timer_start
```

timer_start

```
void timer_start(timer_handle_t handle);
```

描述: 启动 (或重启) 定时器。remaining 重新装填为 interval, 挂入 active_list 头部。

示例:

```
// TCP 发出 SYN 后, 启动重传定时器:
tcp_connect(struct tcp_pcb *pcb, uint32_t remote_ip, uint16_t remote_
{
    pcb->state = TCP_STATE_SYN_SENT;
    tcp_send_segment(pcb, TCP_FLAG_SYN, pcb->snd_una, 0, NULL);

    // 启动 500ms 倒计时: 如果对方不回复, 500ms 后重传 SYN
    timer_start(pcb->retransmit_timer);
    return 0;
}

// TCP 收到 ACK 后, 取消定时器:
timer_stop(pcb->retransmit_timer);

// 下次发新数据时, 重新启动:
timer_start(pcb->retransmit_timer); // remaining 从 500 重新倒数
```

timer_stop

```
void timer_stop(timer_handle_t handle);
```

描述: 从 active_list 摘下定时器, 不归还池。句柄保留, 可后续 restart 或 delete。

示例:

```
// TCP 收到对方的 ACK—数据已确认, 不需要重传了
if (flags & TCP_FLAG_ACK) {
    timer_stop(pcb->retransmit_timer); // 停止倒计时
    pcb->retransmit_count = 0;          // 重置重传计数
    // 定时器还在, 下次发包时 timer_start 继续用
}
```

timer_delete

```
void timer_delete(timer_handle_t handle);
```

描述: 停止定时器并归还池。句柄从此失效——不要再使用。

示例:


```
// TCP 连接关闭—重传定时器不再需要，彻底释放
tcp_pcb_free(struct tcp_pcb *pcb)
{
    if (pcb->retransmit_timer != NULL) {
        timer_delete(pcb->retransmit_timer); // 归还 timer_pool
        pcb->retransmit_timer = NULL;        // 防止悬空指针
    }
    // ... 释放其他资源 ...
}
```

timer_is_active

```
int timer_is_active(timer_handle_t handle);
```

描述：查询定时器是否在倒计时。

返回值：1 激活中，0 未激活

示例：

```
// ARP 模块：发请求前检查是否已有相同请求在进行
if (timer_is_active(arp_timer)) {
    // 已经发过请求了，不等重复
    return;
}
// 没在等—发 ARP 请求，启动定时器
send_arp_request(ni, target_ip);
timer_start(arp_timer);
```

timer_set_interval

```
void timer_set_interval(timer_handle_t handle, uint32_t interval);
```

描述：修改定时周期。只改 `interval` 字段，不影响当前这一轮的 `remaining`。下次 `timer_start` 或 PERIODIC 自动 reload 时生效。

示例：

```
// TCP RTT 探测发现网络变快了，把重传超时从 500ms 缩短到 200ms
uint32_t rtt = measure_rtt();
if (rtt < 100) {
    // 网络很快，缩短超时
    timer_set_interval(pcb->retransmit_timer, 200);
    // 如果当前正在倒计时，不影响剩下时间——只影响下次
}
```

timer_get_remaining

```
uint32_t timer_get_remaining(timer_handle_t handle);
```

描述：查询距离到期还剩多少毫秒。

返回值：剩余毫秒数，未激活返回 0

示例：

```
// 诊断：看看 TCP 重传定时器还剩多久
uint32_t left = timer_get_remaining(pcb->retransmit_timer);
if (left < 50) {
    printf("警告：重传即将触发，剩余 %ums\n", left);
}
```

timer_tick

```
void timer_tick(uint32_t elapsed_ms);
```

描述：定时器驱动函数——遍历 active_list，每个激活定时器的 remaining 减 elapsed_ms。到期则触发回调：ONCE 归池（除非回调重新启动了它），PERIODIC 自动 reload。

示例：

```
// 主循环每 1 秒调一次——调用频率决定定时精度
while (running) {
    tap_poll(g_netif);
    timer_tick(1000); // 所有定时器 remaining -= 1000ms
    usleep(10000);
}

// 也可以用 10ms 精度（主循环够快的话）：
while (running) {
    tap_poll(g_netif);
    timer_tick(10); // 每 10ms 减一次，精度 10ms
    usleep(1000);
}
```

4. netif — 网络接口抽象层

管理所有网络接口，用全局单向链表串起来。对上层提供统一收发接口。

全局链表：netif_list → [eth0] → [wlan0] → [tap0] → NULL
 发送：上层 → netif_find_by_ip → ni->send → 驱动 → 网线
 接收：网线 → 驱动 → netif_input → ni->input → ll_dispatch → 上层协议

netif_init

```
void netif_init(struct netif *ni, const char *name,
                netif_send_fn send, netif_input_fn input);
```

描述：初始化一个 netif 结构体。memset 清零所有字段，填入名称和收发回调。**此时还未注册**，协议栈看不到它。

示例：

```
// driver_register 内部调用 netif_init 填充结构体:
struct netif *driver_register(const char *name, const struct driver_ops *ops,
                             void *priv)
{
    struct netif *ni = &netif_pool[netif_pool_used++];

    // send 直接用驱动提供的发送函数, input 暂为 NULL—稍后在 main 里设置
    netif_init(ni, name, ops->send, NULL);
    ni->ops = ops;        // 存驱动操作表 (init/close 等)
    ni->priv = priv;      // 驱动私有数据 (TAP 的 fd)
    netif_register(ni);   // 挂到全局链表
    return ni;
}
```

netif_register

```
int netif_register(struct netif *ni);
```

描述：把网卡挂到全局链表尾部。注册后协议栈可以通过路由查找到它。先注册的在前——`netif_get_default` 返回的就是第一个。

返回值：0 成功，-1 该指针已在链表中（重复注册）

示例：

```
// 注册两张网卡, eth0 先注册所以是默认网卡
struct netif eth0, wlan0;
netif_register(&eth0);
netif_register(&wlan0);
// netif_list: [eth0] → [wlan0] → NULL
// netif_get_default() → eth0
```

netif_unregister

```
void netif_unregister(struct netif *ni);
```

描述：从全局链表摘除网卡。拔网线或驱动卸载时调用。摘除后再调用 `netif_find_by_ip` 不会返回它。

示例：

```
// 网卡下线—协议栈后续发包不会再选它
void netif_set_down(struct netif *ni)
{
    ni->flags &= ~NETIF_FLAG_UP;
    // netif_find_by_ip 看到 flags 没有 UP, 会跳过此网卡
}

void netif_unregister(struct netif *ni)
{
    // 彻底摘除—比 set_down 更彻底
    // ... 从链表删除 ...
    ni->next = NULL;
}
```

netif_find_by_ip

```
struct netif *netif_find_by_ip(uint32_t ip_addr);
```

描述：根据目标 IP 的子网匹配网卡。遍历链表，用（目标IP & 掩码）==（网卡IP & 掩码）找同一子网的网卡。跳过 netmask==0（未配置）和 flags 不含 UP 的网卡。

返回值：匹配的网卡指针，找不到返回 NULL

示例：

```
// 三张网卡：eth0(10.0.0.1/24), wlan0(192.168.1.5/24), tap0(未配IP)
struct netif *ni;

ni = netif_find_by_ip(0x6400000A); // 10.0.0.100
// → eth0: (100 & 255) = 10.0.0.0, eth0的子网 = 10.0.0.0 ✓

ni = netif_find_by_ip(0x6401A8C0); // 192.168.1.100
// → eth0: x → wlan0: ✓ → 返回 wlan0

ni = netif_find_by_ip(0x08080808); // 8.8.8.8
// → eth0: x → wlan0: x → 返回 NULL → 走默认网关
```

netif_get_default

```
struct netif *netif_get_default(void);
```

描述：返回全局链表头（第一个注册的网卡）。

示例：

```
// ip_output 中路由找不到时, fallback 到默认网卡
ni = netif_find_by_ip(dst_addr);
if (ni == NULL) {
    ni = netif_get_default(); // 没匹配的路由, 走默认
}
if (ni == NULL) {
    // 一张网卡都没有! 丢弃
    mbuf_free(m);
    return -1;
}
```

netif_input

```
int netif_input(struct netif *ni, struct mbuf *m);
```

描述：驱动收到以太网帧后调这个函数把包注入协议栈。内部调 `ni->input` 回调。如果 input 未设置 (NULL)，自动释放 mbuf 防泄漏。

返回值：input 回调的返回值

示例：

```
// TAP 驱动收到帧后的完整路径：
void tap_poll(struct netif *ni)
{
    uint8_t raw[2048];
    ssize_t n = read(fd, raw, sizeof(raw));
    if (n <= 0) return;

    struct mbuf *m = mbuf_alloc_large((uint16_t)n);
    mbuf_append(m, (uint16_t)n);
    mbuf_copy_from(m, raw, (uint16_t)n);

    // 注入协议栈—之后就是 ll_dispatch → ip_input/arp_input → ...
    // 这一行是整个收包路径的唯一入口
    netif_input(ni, m);
}
```

netif_set_up / netif_set_down

```
void netif_set_up(struct netif *ni);
void netif_set_down(struct netif *ni);
```

描述：启用/停用网卡（置位/清除 NETIF_FLAG_UP）。`netif_find_by_ip` 会跳过 DOWN 的网卡。

示例：

```
// main() 中 TAP 初始化完成后:
g_netif = driver_register("tap0", &tap_ops, NULL);
driver_init_all();           // → tap_init → 网卡可用了
netif_set_up(g_netif);      // 启用—路由查找现在能用它了

// 拔网线时:
netif_set_down(g_netif);    // 停用—发包不走它
```

5. driver — 驱动程序适配层

把 netif 和具体的驱动操作绑定在一起。提供以太网帧头解析/封装工具。

driver_register

```
struct netif *driver_register(const char *name, const struct driver_ops *ops,
                             void *priv);
```

描述：从静态池（最多 4 张网卡）取一个 netif，绑定驱动操作接口，注册到全局链表。这是 netif_init + netif_register 的组合包装。

返回值：成功返回 netif 指针，池满（4 张）返回 NULL

示例：

```
// main() 中注册 TAP 网卡:
struct netif *g_netif = driver_register("tap0", &tap_ops, NULL);
if (g_netif == NULL) {
    fprintf(stderr, "注册失败! 池满了 (最多 4 张网卡) \n");
    return 1;
}
// 现在 g_netif 已经在全局链表里了, 但网卡硬件还没初始化
```

driver_unregister

```
void driver_unregister(struct netif *ni);
```

描述：调 `ops->close` 关硬件，从全局链表摘除，归还池槽位。

示例：

```
// 程序退出时：
printf("Shutting down...\n");
driver_unregister(g_netif);
// 内部：tap_ops.close(g_netif) → close(fd)
//      netif_unregister(g_netif) → 从链表摘除
//      netif_pool_used-- → 槽位归还
printf("Goodbye.\n");
```

driver_get_by_name

```
struct netif *driver_get_by_name(const char *name);
```

描述：按接口名称查找网卡。遍历全局链表用 strcmp 匹配。

返回值：找到的 netif 指针，未找到返回 NULL

示例：

```
struct netif *tap = driver_get_by_name("tap0");
if (tap != NULL) {
    printf("tap0 的 IP: %08x\n", ip_get_address(tap));
}
```

driver_init_all

```
void driver_init_all(void);
```

描述：遍历已注册网卡，逐个调 ops->init。init 失败的网卡自动 netif_unregister 摘除。保存 next 指针防止 unregister 后链表断裂。

示例：

```
// 注册完所有网卡后，统一初始化：
driver_register("tap0", &tap_ops, NULL);
driver_register("eth0", &eth_ops, NULL);
driver_init_all();
// → tap_ops.init(tap0) → open("/dev/net/tun") → ioctl(TUNSETIFF)
// → eth_ops.init(eth0) → ...
// 如果某个 init 失败，该网卡被自动摘除，不影响其他
```


eth_header_parse

```
int eth_header_parse(struct mbuf *m, struct eth_header *eth_hdr);
```

描述：从 mbuf 中读取 14 字节以太网帧头。

返回值：0 成功，-1 m->len < 14

示例：

```
// 收包时解析帧头来判断该交给谁
struct eth_header hdr;
if (eth_header_parse(m, &hdr) != 0) {
    mbuf_free(m);
    return -1;
}
printf("收到帧: src=%02x:%02x... type=0x%04x\n",
       hdr.src_mac[0], hdr.src_mac[1], hdr.ether_type);
// hdr.ether_type == 0x0800 → IPv4 → ip_input
// hdr.ether_type == 0x0806 → ARP → arp_input
```

eth_header_add

```
int eth_header_add(struct mbuf *m, const uint8_t *dst_mac,
                  const uint8_t *src_mac, uint16_t ether_type);
```

描述：在 mbuf 前 prepend 14 字节并填入以太网帧头。

返回值：0 成功，-1 headroom 不够 14 字节

示例：

```
// ip_output 完成 IP 头后，调 eth_header_add 封装完整帧：
uint8_t dst_mac[6], src_mac[6] = {0x00, 0x11, 0x22, 0x33, 0x44, 0x00};
// ... ARP 查到了 dst_mac ...

if (eth_header_add(m, dst_mac, src_mac, ETHERTYPE_IPV4) != 0) {
    // headroom 不够了——前面栈太深。检查 mbuf 分配逻辑
    mbuf_free(m);
    return -1;
}
// 现在 m 是以太网帧 [ETH 14B|IP 头|负载]，可以发送了
ni->send(ni, m);
```

eth_addr_cmp

```
int eth_addr_cmp(const uint8_t *a, const uint8_t *b);
```

描述：比较 6 字节 MAC 地址。

返回值：1 相等，0 不相等（或任一为 NULL）

示例：

```
// 判断收到的帧是不是发给我们自己的
if (eth_addr_cmp(hdr.dst_mac, ni->hwaddr) ||
    eth_addr_cmp(hdr.dst_mac, broadcast_mac)) {
    // 是发给我的（单播或广播），处理它
    process_frame(ni, m);
} else {
    // 不是给我的，在混杂模式下抓到而已，丢弃
    mbuf_free(m);
}
```

eth_addr_from_str

```
int eth_addr_from_str(const char *str, uint8_t *addr);
```

描述：MAC 字符串 "AA:BB:CC:DD:EE:FF" 转为 6 字节数组。

返回值：0 成功，-1 格式错误（长度不足、分隔符不对、非 hex 字符）

示例：

```
// 从配置文件读取 MAC 地址
uint8_t mac[6];
if (eth_addr_from_str("00:11:22:33:44:55", mac) != 0) {
    fprintf(stderr, "MAC 格式错误\n");
    return -1;
}
memcpy(ni->hwaddr, mac, 6); // 把解析出来的 MAC 设到网卡上
```

6. tap — TAP 虚拟网卡驱动

基于 Linux `/dev/net/tun` 创建虚拟以太网卡。程序通过文件描述符收发原始以太网帧。Wireshark 抓 tap0 可以观察所有交互。

tap_ops (全局对象)

```
extern const struct driver_ops tap_ops;
```

描述：TAP 驱动的操作接口表，包含 `tap_init`、`tap_send`、`tap_close`。传给 `driver_register` 使用。

示例：

```
// 注册时只需传 &tap_ops, driver_init_all 内部自动调这三个函数:  
// tap_init → open("/dev/net/tun") → ioctl → 告诉内核创建 tap0  
// tap_send → write(fd, 帧) → 内核收到 → Wireshark 可见  
// tap_close → close(fd) → 内核删除 tap0 接口  
driver_register("tap0", &tap_ops, NULL);  
driver_init_all();  
// 现在在另一个终端:  
// sudo ip link set tap0 up  
// sudo ip addr add 10.0.0.2/24 dev tap0  
// ping 10.0.0.1 → tap0 上可以看到 ICMP 交互
```

7. ll_dispatch — 链路层分发器

按照以太网帧类型 (EtherType) 分发给注册的协议处理器。这是 netif 和 IP/ARP 之间的胶水层。

ll_dispatch_register

```
int ll_dispatch_register(uint16_t ether_type, ll_handler_fn handler);
```

描述：注册协议处理器。同一 ether_type 只能注册一个 handler。

返回值：0 成功，-1 类型已被占用或表满 (16)

示例：

```
// 各模块 init 时注册自己:
// ip_init → ll_dispatch_register(0x0800, ip_input);
// arp_init → ll_dispatch_register(0x0806, arp_input);
// 现在分发表:
//   [0] 0x0800 → ip_input   (IPv4 包交给 IP 层)
//   [1] 0x0806 → arp_input  (ARP 包交给 ARP 模块)
//   [2..15] 空

// 如果误注册两次同一个类型:
if (ll_dispatch_register(0x0800, some_func) == -1) {
    // 0x0800 已经被 ip_input 占了
}
```

ll_dispatch_unregister

```
void ll_dispatch_unregister(uint16_t ether_type);
```

描述：注销处理器。之后再收到该类型帧直接丢弃。

示例：

```
// 卸载 IPv6 支持——之后收到 0x86DD 的包直接丢弃
ll_dispatch_unregister(ETHERTYPE_IPV6);
```

ll_dispatch_input

```
int ll_dispatch_input(struct netif *ni, struct mbuf *m);
```

描述：收包分发入口——整个接收路径经过这里分流。读帧头 ether_type → 查注册表 → 从 IP 包学 MAC → 剥帧头 → 调对应 handler。

示例：

```
// 收到一个完整以太网帧后, ll_dispatch_input 做什么:
// 帧: [dst_mac 6B|src_mac 6B|type 0x0806|ARP 包 28B]
//                                     ↑ data
int ll_dispatch_input(struct netif *ni, struct mbuf *m)
{
    uint16_t ether_type = (m->data[12] << 8) | m->data[13];
    // ether_type = 0x0806 (ARP)

    if (ether_type == ETHERTYPE_IPV4) {
        // 从 IP 包学习源 MAC: 如果这个包来自 10.0.0.2,
        // 把 10.0.0.2 → src_mac 记进 ARP 表, 下次直接发包
        arp_learn(ni, src_ip_from_packet, m->data + 6);
    }

    // 查注册表: 0x0806 → arp_input
    mbuf_pull(m, 14); // 剥帧头
    return arp_input(ni, m);
    // arp_input 看到的是纯 ARP 包, 不包含以太网帧头
}
```

8. arp — 地址解析协议

把 IPv4 地址翻译成 MAC 地址。维护 32 条 ARP 缓存。

arp_init

```
void arp_init(void);
```

描述: 清空 ARP 表, 向 ll_dispatch 注册 (EtherType=0x0806)。

示例:

```
arp_init();
// ARP 表 32 条全部 FREE
// ll_dispatch: 0x0806 → arp_input (和 ip_init 注册的 0x0800→ip_input)
```

arp_input

```
int arp_input(struct netif *ni, struct mbuf *m);
```

描述：处理收到的 ARP 包——两分支。

- **收到 ARP 请求**（别人问"谁是 X.X.X.X?"）：如果问的是我们的 IP → 在原地改写为回复 → 发给对方 → 顺手学习对方的 IP→MAC。
- **收到 ARP 回复**（有人回答我们的询问）：更新 PENDING 表项为 RESOLVED → 把挂起队列里的包全部发出去。

返回值：0 成功，-1 不是发给本机/不支持的类型

示例：

```
// Ping 10.0.0.1 的完整 ARP 交互：
// 1. 内核广播 ARP 请求："谁是 10.0.0.1?"
//    ll_dispatch → arp_input
//    arp_input: target_ip=10.0.0.1 == ni->ip_addr → 是问我的!
//    → 改 opcode: REQUEST→REPLY
//    → 改 sender=我的MAC/IP, target=对方的MAC/IP
//    → eth_header_add → ni->send → 内核收到回复
//    → 顺手 arp_learn(10.0.0.2, 对方MAC)
//
// 2. 内核收到 ARP 回复, 知道我们的 MAC 了
// 3. 内核发 ICMP Echo Request → ip_input → icmp_input → 回复 pong
```

arp_query

```
int arp_query(struct netif *ni, uint32_t ip_addr, uint8_t *mac_out,
              struct mbuf *m);
```

描述：查询 IP 对应的 MAC——IP 层发包前必调。三条路径：

- **已解析 (RESOLVED/STATIC)** → 直接填 mac_out, 返回 0。调用者自己封装帧头发送。
- **PENDING 中** → m 加入挂起队列（不重复广播），返回 -1。调用者不用管 m。
- **没找到** → 新建 PENDING 表项 → 广播 ARP 请求 → m 挂起，返回 -1。

返回值：0 已解析（mac_out 有效），-1 未命中（m 已被 ARP 接管）

示例：

```
// ip_output 中的标准调用模式：
int ip_output(uint32_t dst_addr, uint8_t proto, struct mbuf *m)
{
    uint8_t dst_mac[6];
    int ret = arp_query(ni, next_hop, dst_mac, m);
    if (ret == 0) {
        // MAC 已知，直接封装发送：
        eth_header_add(m, dst_mac, ni->hwaddr, ETHERTYPE_IPV4);
        return ni->send(ni, m);
    }
    // ret == -1: m 已被 ARP 挂起，解析完成后 ARP 模块会自动发送。
    // 调用者不需要做任何事—m 的所有权已转移。
    return -1;
}
```

arp_add_static

```
int arp_add_static(struct netif *ni, uint32_t ip_addr, const uint8_t
```

描述：手工添加静态 ARP 表项（STATIC 状态，永不过期）。

返回值：0 成功，-1 表满（32 条全占用）

示例：

```
// 网关 MAC 很少变，写死省掉每次 ARP 查询
uint8_t gw_mac[6] = {0x00, 0x11, 0x22, 0x33, 0x44, 0x55};
arp_add_static(&tap0, 0x0100000A, gw_mac); // 10.0.0.1 网关→MAC
// 之后发给 10.0.0.1 不再需要 ARP 广播
```

arp_remove / arp_remove_by_netif

```
void arp_remove(uint32_t ip_addr);
void arp_remove_by_netif(struct netif *ni);
```

描述：arp_remove 删除指定 IP 的表项并释放挂起队列；arp_remove_by_netif 删除指定网卡的所有关联表项（网卡注销时调用）。

示例：

```
// 网卡下线—清理所有相关 ARP 缓存
arp_remove_by_netif(&eth0);
// 所有通过 eth0 学到的 IP→MAC 映射都清除
```

arp_learn

```
void arp_learn(struct netif *ni, uint32_t ip_addr, const uint8_t *mac)
```

描述：从收到的 IP 包中被动学习 IP→MAC 映射（RESOLVED 状态）。ll_dispatch 收到任何 IP 包时自动调用，无需用户干预。

示例：

```
// ll_dispatch_input 收到 IP 包时自动调用 arp_learn:
// 内核 10.0.0.2 发来一个包，帧头里有它的 MAC ea:73:f9:ae:f1:6d
// 自动学到：10.0.0.2 → ea:73:f9:ae:f1:6d
// 下次 NanoLink 给 10.0.0.2 发包就不需要 ARP 了，直接用这个 MAC
```

arp_tick

```
void arp_tick(void);
```

描述：定期维护 ARP 表。PENDING 条目超时重试 3 次，超时后释放挂起队列并回收条目。

示例：

```
// 主循环每秒调用—驱动 ARP 超时重试
// 第 1 秒：PENDING 表项 retry_count=1 → 重发 ARP 请求
// 第 2 秒：retry_count=2 → 再重发
// 第 3 秒：retry_count=3 → ≥3 → 超时放弃！释放所有挂起的包
while (running) {
    arp_tick(); // 维护 ARP 缓存
    sleep(1);
}
```

9. ip — IPv4 网络层

协议栈核心层——收包校验分发、发包路由封装。

ip_init

```
void ip_init(void);
```


描述：清空上层协议表（8 槽位）和分片表（8 槽位），向 ll_dispatch 注册（EtherType=0x0800）。

示例：

```
ip_init();
// 上层协议表全空，分片表全空
// ll_dispatch: 0x0800 → ip_input
```

ip_register_proto / ip_unregister_proto

```
int ip_register_proto(uint8_t proto, ip_proto_handler_fn handler);
void ip_unregister_proto(uint8_t proto);
```

描述：注册/注销上层协议处理器。ICMP(1)、TCP(6)、UDP(17) 启动时各注册自己的 handler。

返回值：register 成功返回 0，协议号被占用或表满返回 -1

示例：

```
// 各协议模块 init 时注册：
icmp_init() → ip_register_proto(IPPROTO_ICMP, icmp_input); // 1→icmp
tcp_init() → ip_register_proto(IPPROTO_TCP, tcp_input); // 6→tcp
udp_init() → ip_register_proto(IPPROTO_UDP, udp_input); // 17→udp

// 注册表：
// [0] proto=1 → icmp_input
// [1] proto=6 → tcp_input
// [2] proto=17 → udp_input
// [3..7] 空

// ip_input 收到包后根据 IP 头的 proto 字段查表分发
```

ip_set_address / ip_get_address

```
void ip_set_address(struct netif *ni, uint32_t ip_addr, uint32_t netmask);
uint32_t ip_get_address(struct netif *ni);
```

描述：为网卡配置/获取 IPv4 地址和掩码。地址均为网络字节序。

示例：

```
// main() 中给 tap0 配 IP:
ip_set_address(g_netif, ip_addr, netmask);
// g_netif->ip_addr = 10.0.0.1 (网络字节序)
// g_netif->netmask = 255.255.255.0 (网络字节序)

// arp_input 中用 ip_get_address 对比 target_ip:
if (target_ip != ip_get_address(ni)) {
    // 不是问我的, 忽略
}
```

ip_is_local

```
int ip_is_local(uint32_t ip_addr);
```

描述：检查 IP 是否属于本机任一网卡。ip_input 中用于过滤不是发给我们的包。

返回值：1 是本机，0 不是

示例：

```
// 收到 IP 包后第一件事：是发给我的吗？
if (!ip_is_local(iph->dst_addr)) {
    // 不是我的——NanoLink 暂不支持路由转发，直接丢弃
    mbuf_free(m);
    return -1;
}
```

ip_input

```
int ip_input(struct netif *ni, struct mbuf *m);
```

描述：IP 包输入处理——校验 → 查本地 → 剥头 → 分发。全路径：

1. 检查版本（必须是 IPv4）
2. 检查头长度
3. 校验 checksum
4. 检查是否是发给本机
5. 检查是否分片（暂不支持重组，直接丢弃）
6. 按 proto 字段查表，调上层 handler

返回值：上层 handler 的返回值，-1 表示包被丢弃

示例：

```
// 一个 UDP 包到达时的处理链:
// m->data 指向: [IP头 20B: ver=4, proto=17, src=10.0.0.2, dst=10.0.0.1]

ip_input(&tap0, m):
  1. ver=4 ✓
  2. header_len=20, 合法 ✓
  3. checksum 验证通过 ✓
  4. dst=10.0.0.1, ip_is_local()==1 ✓
  5. 无分片 ✓
  6. proto=17 → 查表 → udp_input
  7. mbuf_pull(m, 20) → data 指向 UDP 头
  8. udp_input(&tap0, m, src=10.0.0.2, dst=10.0.0.1)
```

ip_output

```
int ip_output(uint32_t dst_addr, uint8_t proto, struct mbuf *m);
```

描述：发送 IPv4 数据包——传输层统一入口。内部完成：路由查找（含网关）→ mbuf_prepend 加 IP 头 → 填字段 → 校验和 → arp_query 查 MAC → eth_header_add → ni->send。

返回值：0 成功发送，-1 路由失败/ARP 未解析

示例：

```
// UDP 层发送 100 字节数据给 10.0.0.2:8080:
// m: [UDP头 8B|数据 100B] (UDP 层已 prepend 好)
ip_output(0x0200000A, IP_PROTO_UDP, m);
// 内部流程:
// 1. route_lookup(10.0.0.2) → 直连 tap0, next_hop=10.0.0.2
// 2. mbuf_prepend(m, 20) → [IP头 20B|UDP头 8B|数据 100B]
// 3. IP头: ver=4, src=10.0.0.1, dst=10.0.0.2, proto=17
// 4. checksum 计算
// 5. arp_query(10.0.0.2, dst_mac, m)
//    → 已解析(RESOLVED) → 拿到 MAC
// 6. eth_header_add → [ETH 14B|IP 20B|UDP 8B|数据 100B]
// 7. tap0.send → write(fd) → 内核收到 → 发给 10.0.0.2 的 8080 端口
```

ip_frag_tick

```
void ip_frag_tick(void);
```

描述：清理超时的分片重组条目。主循环每秒调用。

示例：

```
while (running) {  
    ip_frag_tick(); // 分片 15 秒没到齐 → 丢弃已收集的片段  
}
```

10. icmp — 互联网控制消息协议

实现 Ping 回复 (Echo Request → Echo Reply) 和差错报告。

icmp_init

```
void icmp_init(void);
```

描述：向 IP 层注册 icmp_input (协议号=1)。

示例：

```
icmp_init();  
// ip_proto_table: proto=1 → icmp_input
```

icmp_input

```
int icmp_input(struct netif *ni, struct mbuf *m, uint32_t src_ip, uir
```

描述：处理收到的 ICMP 包。目前支持 Echo Request (type=8, ping) → 原地改为 Echo Reply (type=0)，重算 checksum，调用 ip_output 发回。

参数：ni — 收包网卡； m — ICMP 包 (IP 头已移)； src_ip / dst_ip — 源/目标 IP 地址

返回值：0 成功，-1 未知类型/checksum 错

示例：

```
// Ping 的回复流程—零拷贝，直接在同一个 mbuf 上改：
// 收到：type=8(ECHO_REQUEST), code=0, checksum=..., data="ping 的 56
// ↓
// icmp_input:
//   1. 校验 checksum ✓
//   2. icmp->type = 0 (ECHO_REPLY)      ← 原地改!
//   3. icmp_checksum(..., mode=1)      ← 重算 checksum
//   4. ip_output(src_ip, IP_PROTO_ICMP, m) ← 原 mbuf 发回去
// ↓
// 对方收到：type=0(ECHO_REPLY), checksum正确 → ping 成功!
```

icmp_send_error

```
int icmp_send_error(uint8_t type, uint8_t code, uint32_t info, struct
```

描述：发送 ICMP 差错报文（如"端口不可达"）。报文中携带触发差错的原始 IP 包头。

示例：

```
// UDP 收到包但端口没人监听—发送端口不可达：
// original = 收到的原始 IP 包
icmp_send_error(ICMP_TYPE_DEST_UNREACHABLE,
                ICMP_CODE_PORT_UNREACHABLE, 0, original);
// 对方收到：ICMP Destination Unreachable (Port Unreachable)
```

11. route — 路由表

最长前缀匹配查出口网卡和下一跳。

route_init / route_add / route_remove

```
void route_init(void);
int route_add(uint32_t dest, uint32_t netmask, uint32_t gateway, struct
void route_remove(uint32_t dest, uint32_t netmask);
```

描述：初始化/添加/删除路由条目。gateway=0 表示直连。

示例：

```

route_init();

// main() 中的路由设置:
// 1. 直连路由: 同网段的包直接发给目标
route_add(ip_addr & netmask, netmask, 0, g_netif);
//    10.0.0.0/24 → tap0 (直连) — 发给 10.0.0.x 的包直接通信

// 2. 默认路由: 找不到具体路由时走这里
uint32_t gw = (ip_addr & netmask) | 0x01000000; // 网关假设是 10.0.0.
route_set_default(gw, g_netif);
//    0.0.0.0/0 → 下一跳 10.0.0.1 → tap0 — 发给 8.8.8.8 的包交给网关

```

route_lookup

```

int route_lookup(uint32_t dst_addr, struct netif **ni_out, uint32_t >

```

描述: 最长前缀匹配查路由。遍历所有条目, 找 (dst&mask)==dest 且 mask 前缀最长的。gateway 路由返回网关地址作为 next_hop, 直连路由返回目标地址本身。

返回值: 0 找到, -1 未找到

示例:

```

// 路由表: 10.0.0.0/24 直连, 0.0.0.0/0 → 10.0.0.1 网关

struct netif *ni;
uint32_t next_hop;

route_lookup(0x6400000A, &ni, &next_hop); // 10.0.0.100
// → 匹配 10.0.0.0/24 (前缀24) ✓, 返回 ni=tap0, next_hop=10.0.0.100 (直

route_lookup(0x08080808, &ni, &next_hop); // 8.8.8.8
// → 10.0.0.0/24 不匹配 x
// → 0.0.0.0/0 匹配 ✓ (前缀0) → 返回 ni=tap0, next_hop=10.0.0.1 (网关)
// arp_query 查的是 next_hop (网关的 MAC), 不是原目标 IP

```

route_set_default / route_dump

```

int route_set_default(uint32_t gateway, struct netif *ni);
void route_dump(void);

```

描述: route_set_default 添加 0.0.0.0/0 默认路由。route_dump 打印全部路由表。

示例：

```
route_set_default(0x0100000A, &tap0);
route_dump();
// 输出：
//   Route Table:
//     [0] dest=0a000000 mask=ffffff00 gw=00000000 flags=01
//     [1] dest=00000000 mask=00000000 gw=0a000001 flags=0a
```

12. udp — 用户数据报协议

基于端口的数据报收发。PCB 链表管理所有绑定端口。

udp_init

```
void udp_init(void);
```

描述：向 IP 层注册 udp_input（协议号=17）。

示例：

```
udp_init();
// ip_proto_table: proto=17 → udp_input
```

udp_new

```
struct udp_pcb *udp_new(void);
```

描述：从堆分配 UDP 控制块（日后可改为静态池），加入全局 PCB 链表。

返回值：PCB 指针，堆分配失败返回 NULL

示例：

```
// socket_create(SOCK_DGRAM) 内部调 udp_new:
struct socket *socket_create(int type)
{
    struct udp_pcb *pcb = udp_new();
    if (pcb == NULL) {
        return NULL; // 内存不够
    }
    sock->pcb = pcb;
    // 现在这个 socket 有了 UDP 控制块, 可以 bind/send 了
}
```

udp_bind

```
int udp_bind(struct udp_pcb *pcb, uint32_t local_ip, uint16_t port);
```

描述: 绑定本地端口。ip=0 表示 INADDR_ANY (监听所有接口)。

返回值: 0 成功, -1 端口被占用

示例:

```
// UDP echo server 绑定 8888:
struct udp_pcb *pcb = udp_new();
udp_bind(pcb, 0, 8888); // 监听所有网卡的 8888 端口
// 现在从任何 IP 发到 NanoLink:8888 的 UDP 包都会被这个 PCB 接收
```

udp_connect

```
void udp_connect(struct udp_pcb *pcb, uint32_t remote_ip, uint16_t remote_port);
```

描述: 设置默认远端。之后 `udp_send` 可以不传地址, 且只接收来自该远端的包。

示例:

```
// 指定只跟 10.0.0.2:9090 通信:
udp_connect(pcb, 0x0200000A, 9090);
// 之后:
//  udp_send(pcb, m, 0, 0); // dst_ip=0→用默认 10.0.0.2, dst_port=0→用默认 9090
//  udp_input 只投递来自 10.0.0.2:9090 的包
```


udp_set_rcv_callback

```
void udp_set_rcv_callback(struct udp_pcb *pcb, udp_rcv_fn callback)
```

描述：设置数据到达回调。回调签名：`void callback(struct udp_pcb *, struct mbuf *, uint32_t src_ip, uint16_t src_port)`

示例：

```
// 收到数据时打印来源和内容：
void on_udp_rcv(struct udp_pcb *pcb, struct mbuf *m,
                uint32_t src_ip, uint16_t src_port)
{
    printf("来自 %08x:%d 的 %d 字节数据\n", src_ip, src_port, m->total_len);
    // 回显：
    udp_send(pcb, m, src_ip, src_port);
}
udp_set_rcv_callback(pcb, on_udp_rcv);
```

udp_send

```
int udp_send(struct udp_pcb *pcb, struct mbuf *m, uint32_t dst_ip, uint16_t dst_port)
```

描述：发送 UDP 数据报。内部 `mbuf_prepend` 添加 UDP 头（8 字节），调用 `ip_output`。

参数：`dst_ip/dst_port` 填 0 时使用 `udp_connect` 设置的值

返回值：0 成功，-1 失败

示例：

```
// 发 "Hello" 到 10.0.0.2:8080
struct mbuf *m = mbuf_alloc();
mbuf_append(m, 5); // "Hello" 5 字节
memcpy(m->data, "Hello", 5);

udp_send(pcb, m, 0x0200000A, 8080);
// 内部：
// 1. mbuf_prepend(m, 8) → [UDP头: src=监听端口, dst=8080, len=13 | "Hello"]
// 2. ip_output(10.0.0.2, IPPROTO_UDP, m)
```

udp_remove

```
void udp_remove(struct udp_pcb *pcb);
```

描述：从全局链表摘除并 osal_free 控制块。

示例：

```
// socket_close 中释放 PCB:
if (sock->pcb != NULL) {
    udp_remove((struct udp_pcb *)sock->pcb);
    sock->pcb = NULL;
}
```

udp_input

```
int udp_input(struct netif *ni, struct mbuf *m, uint32_t src_ip, uint16_t src_port, uint16_t dst_port);
```

描述：UDP 包输入处理。按目标端口匹配 PCB → 剥 UDP 头 → 调 recv_callback 投递。无匹配端口时丢弃 (TODO: ICMP 端口不可达)。

示例：

```
// 收到 10.0.0.2:54321 → 10.0.0.1:8888 的 UDP 包:
// m: [UDP头: src=54321, dst=8888, len=50 | 42字节数据]
udp_input(&tap0, m, 0x0200000A, 0x0100000A):
1. dst_port = 8888
2. 遍历 PCB 链表 → 找到 local_port=8888 的 PCB ✓
3. 检查 remote 限制 → 无限制或匹配 ✓
4. mbuf_pull(m, 8) → 剥掉 UDP 头
5. pcb->recv_callback(pcb, m, src_ip=10.0.0.2, src_port=54321)
```

13. socket — Socket 抽象层

提供类 BSD socket API。通过 transport_ops 统一 UDP/TCP 差异。

socket_init

```
void socket_init(void);
```

描述：把 socket_pool[16] 串成空闲链表。

示例：

```
socket_init();  
// 16 个 socket 待命
```

socket_create

```
struct socket *socket_create(int type);
```

描述：创建 socket。SOCK_DGRAM 自动创建 UDP PCB 并设置 recv_callback；SOCK_STREAM 暂创建空壳。

返回值：socket 指针，16 个全部分配时返回 NULL

示例：

```
// 创建 UDP socket——自动分配 UDP PCB:  
struct socket *sock = socket_create(SOCK_DGRAM);  
if (sock == NULL) {  
    printf("16 个 socket 全部用完了\n");  
    return -1;  
}  
// sock->pcb 已指向一个 UDP PCB,  
// sock->recv_queue 是数据接收队列
```

socket_close

```
void socket_close(struct socket *sock);
```

描述：关闭 socket——释放 recv_queue 所有数据、释放 PCB（UDP 调用 udp_remove）、归还 socket 池。

示例：

```
// 程序退出时清理：  
socket_close(sock);  
// 内部：  
// 1. 释放 recv_queue 上所有未读数据  
// 2. SOCK_DGRAM → udp_remove(pcb)  
// 3. sock->state = FREE, 归还 socket_pool
```

socket_bind

```
int socket_bind(struct socket *sock, const struct sockaddr_in *addr);
```

描述：绑定本地地址和端口。

返回值：0 成功，-1 端口被占用

示例：

```
struct sockaddr_in addr;  
addr.sin_family = AF_INET;  
addr.sin_port   = 8888;    // 监听端口  
addr.sin_addr   = 0;      // INADDR_ANY  
memset(addr.sin_zero, 0, sizeof(addr.sin_zero));  
socket_bind(sock, &addr);
```

socket_connect

```
int socket_connect(struct socket *sock, const struct sockaddr_in *addr);
```

描述：连接远端。UDP 仅设置默认目标；TCP 发起三次握手（暂未实现）。

返回值：0 成功，-1 失败

示例：

```
struct sockaddr_in addr = {AF_INET, 9090, 0x0200000A}; // 10.0.0.2:9090  
socket_connect(sock, &addr);  
// 之后：  
//  socket_send(sock, data, len, NULL); // addr=NULL 时用 connect 设  
//  收到的包自动 filter 只接收来自 10.0.0.2:9090 的
```

socket_send

```
int socket_send(struct socket *sock, const void *data, uint16_t len,  
               const struct sockaddr_in *addr);
```

描述：发送数据。内部: mbuf_alloc() → mbuf_append() → mbuf_copy_from() → 调对应协议的 send。

返回值：实际发送字节数，-1 失败

示例：

```
// 发 "Hello" 给 10.0.0.2:8080
struct sockaddr_in addr = {AF_INET, 8080, 0x0200000A};
int n = socket_send(sock, "Hello", 5, &addr);
if (n == 5) {
    printf("发送成功\n");
} else {
    printf("发送失败\n");
}
```

socket_recv

```
int socket_recv(struct socket *sock, void *buf, uint16_t len,
                struct sockaddr_in *addr);
```

描述：从内部 recv_queue 取数据拷到用户 buf。非阻塞。addr 不为 NULL 时填入发送方地址。

返回值：>0 实际读取字节数，0 暂无数据可读，-1 连接已关闭或参数错误

示例：

```
// 主循环中轮询接收：
char buf[1024];
struct sockaddr_in from;
int n = socket_recv(sock, buf, sizeof(buf), &from);
if (n > 0) {
    printf("收到 %d 字节, 来自端口 %d\n", n, from.sin_port);
    // 回显：
    socket_send(sock, buf, n, &from);
} else if (n == 0) {
    // 当前无数据, 稍后重试
} else {
    // n == -1: 连接已关闭
}
```

14. tcp — 传输控制协议

面向连接、可靠的字节流传输。TCP 状态机覆盖三次握手、数据确认、重传、四次挥手全过程。

tcp_init

```
void tcp_init(void);
```

描述：把 tcp_pcb_pool[16] 串成空闲链表，向 IP 层注册 tcp_input（协议号=6）。

示例：

```
tcp_init();  
// PCB 池：16 个 FREE  
// ip_proto_table: proto=6 → tcp_input
```

tcp_new

```
struct tcp_pcb *tcp_new(void);
```

描述：从池中分配 PCB，初始状态为 CLOSED。自动创建重传定时器。

返回值：PCB 指针，16 个全部分配时返回 NULL

示例：

```
struct tcp_pcb *pcb = tcp_new();  
if (pcb == NULL) {  
    printf("TCP PCB 池耗尽\n");  
    return -1;  
}  
// pcb->state = CLOSED, pcb->rcv_wnd = 8192, pcb->mss = 536  
// pcb->retransmit_timer 已创建但未启动
```

tcp_bind

```
int tcp_bind(struct tcp_pcb *pcb, uint32_t local_ip, uint16_t port);
```

描述：绑定本地 IP 和端口。填 0 表示 INADDR_ANY。

返回值：0 成功，-1 端口被占用

示例：

```
tcp_bind(pcb, 0x0100000A, 80); // 绑定 10.0.0.1:80  
// 之后可以 listen 或 connect
```

tcp_listen

```
void tcp_listen(struct tcp_pcb *pcb, uint8_t backlog);
```

描述：进入 LISTEN 状态。此时 PCB 只接受新连接，不承载具体数据流。

示例：

```
tcp_listen(pcb, 5);  
// PCB 状态: CLOSED → LISTEN  
// 之后收到 SYN 时, tcp_new_connection 创建子 PCB 来处理该连接
```

tcp_connect

```
int tcp_connect(struct tcp_pcb *pcb, uint32_t remote_ip, uint16_t remote_port, err_t err_callback);
```

描述：主动发起连接（三次握手的发起方）。发送 SYN，进入 SYN_SENT 状态，启动重传定时器。

返回值：0 SYN 已发送，-1 失败

示例：

```
// 客户端连接服务器：  
tcp_bind(pcb, 0, 0); // 先绑定（端口填0让内核分配）  
tcp_connect(pcb, 0x0200000A, 80); // 连接 10.0.0.2:80  
// 之后状态: SYN_SENT → (等 SYN-ACK) → ESTABLISHED  
// 如果失败: err_callback 被调用
```

tcp_accept

```
struct tcp_pcb *tcp_accept(struct tcp_pcb *listen_pcb);
```

描述：从 accept 队列取一个已完成的连接。三次握手完成后子 PCB 自动进入此队列。

返回值：子 PCB，队列空返回 NULL

示例：

```
// 主循环中检查新连接：
struct tcp_pcb *client = tcp_accept(server_pcb);
if (client != NULL) {
    printf("新客户端连接!\n");
    tcp_set_recv_callback(client, handle_client_data);
}
```

tcp_send

```
int tcp_send(struct tcp_pcb *pcb, struct mbuf *m);
```

描述：发送数据。数据加入 unsent 队列，受发送窗口限制。调用者传交 mbuf 所有权——tcp_send 会负责最终释放 m。

返回值：0 成功入队，-1 连接未建立

示例：

```
// TCP echo: 收到什么回什么
void on_recv(struct tcp_pcb *pcb, struct mbuf *m)
{
    if (m != NULL) {
        tcp_send(pcb, m); // m 所有权交给 TCP
        // 不要再访问 m!
    } else {
        printf("对方关闭连接\n"); // m==NULL → FIN
    }
}
```

tcp_close

```
void tcp_close(struct tcp_pcb *pcb);
```

描述：主动关闭连接。ESTABLISHED → FIN_WAIT_1 → ... → TIME_WAIT → CLOSED。

示例：

```
tcp_close(pcb);
// 状态：ESTABLISHED → FIN_WAIT_1 (发 FIN)
//      对方 ACK → FIN_WAIT_2
//      对方 FIN → TIME_WAIT (等 60s 后自动回收)
```


tcp_abort

```
void tcp_abort(struct tcp_pcb *pcb);
```

描述：立即终止连接——发 RST，释放所有资源。不等四次挥手完成。

示例：

```
// 错误处理中暴力关闭连接：  
tcp_abort(pcb); // → 发 RST → tcp_pcb_free  
// 对方收到 RST，立即认为连接断开
```

回调设置

```
void tcp_set_recv_callback(struct tcp_pcb *pcb, tcp_recv_fn cb);  
void tcp_set_accept_callback(struct tcp_pcb *pcb, tcp_accept_fn cb);  
void tcp_set_sent_callback(struct tcp_pcb *pcb, tcp_sent_fn cb);  
void tcp_set_err_callback(struct tcp_pcb *pcb, tcp_err_fn cb);
```

描述：设置四类事件回调。

示例：

```
// 完整的 TCP echo server 例子:

static void echo_recv(struct tcp_pcb *pcb, struct mbuf *m) {
    if (m != NULL) {
        printf("收到 %d 字节, 回显\n", m->total_len);
        tcp_send(pcb, m);    // 回显
    } else {
        printf("客户端关闭连接\n");
    }
}

static void echo_accept(struct tcp_pcb *new_pcb) {
    printf("新客户端\n");
    tcp_set_recv_callback(new_pcb, echo_recv);
}

static void echo_err(struct tcp_pcb *pcb, int err) {
    printf("错误: %d\n", err);
}

// 启动服务器:
struct tcp_pcb *server = tcp_new();
tcp_bind(server, 0x0100000A, 9999);    // 10.0.0.1:9999
tcp_listen(server, 5);
tcp_set_accept_callback(server, echo_accept);
tcp_set_err_callback(server, echo_err);
// 测试: echo "hello" | nc 10.0.0.1 9999
// 终端输出:
//   新客户端
//   收到 6 字节, 回显
//   客户端关闭连接
```

tcp_slow_tick / tcp_fast_tick

```
void tcp_slow_tick(void);
void tcp_fast_tick(void);
```

描述: 慢定时器每秒调用 (驱动重传); 快定时器 200ms 调用 (驱动延迟 ACK)。

示例:

```
// 已集成到 main_loop_tick() 中, 无需手动调用
```

tcp_input

```
int tcp_input(struct netif *ni, struct mbuf *m, uint32_t src_ip, uint32_t dest_ip, uint16_t src_port, uint16_t dest_port);
```

描述：TCP 段输入处理——**核心状态机**。处理 11 种 TCP 状态下的所有事件（SYN/ACK/FIN/RST/数据）。IP 层自动调用，用户不直接使用。

状态转换流程：

```
LISTEN + SYN          → SYN_RECV (发 SYN-ACK)
SYN_RECV + ACK        → ESTABLISHED (握手完成!)
ESTABLISHED + PSH+ACK → recv_callback (数据交给应用)
ESTABLISHED + FIN     → CLOSE_WAIT (对方关了, 等应用关)
FIN_WAIT_1 + ACK      → FIN_WAIT_2
FIN_WAIT_2 + FIN      → TIME_WAIT (等 60s)
LAST_ACK + ACK        → CLOSED (释放 PCB)
```

15. event_bus — 事件总线

发布-订阅模式的事件通知。可选模块，用于模块间松耦合通信。

event_bus_init

```
void event_bus_init(void);
```

描述：清空监听表（16 槽位）。

event_subscribe

```
int event_subscribe(event_type_t type, event_handler_fn handler);
```

描述：订阅事件。一个 handler 只能关联一种类型。

示例：

```
void on_netif_up(const struct event *e) {
    struct netif *ni = (struct netif *)e->source;
    printf("%s 启用了\n", ni->name);
}
event_subscribe(EVENT_NETIF_UP, on_netif_up);
```

event_publish / event_publish_simple

```
void event_publish(const struct event *e);
void event_publish_simple(event_type_t type, void *source, void *data,
                          uint32_t param1, uint32_t param2);
```

描述：发布事件，同步调用所有匹配订阅者的回调。

示例：

```
// 网卡启用后通知订阅者：
event_publish_simple(EVENT_NETIF_UP, &tap0, NULL, 0, 0);
// → 所有订阅 EVENT_NETIF_UP 的回调被调用
```

event_unsubscribe

```
void event_unsubscribe(event_type_t type, event_handler_fn handler);
```

16. main — 应用入口

协议栈启动、主循环、内嵌 ICMP/UDP/TCP echo 测试服务器。

命令行：

```
sudo ./bin/nanolink [接口名] [IP] [掩码]
sudo ./bin/nanolink tap0 10.0.0.1 255.255.255.0
```

启动后另一终端测试：

```
sudo ip link set tap0 up
sudo ip addr add 10.0.0.2/24 dev tap0

ping 10.0.0.1                                # ICMP Echo 回复
echo "hi" | nc -u -w1 10.0.0.1 8888          # UDP Echo (端口 8888)
echo "hi" | nc -w2 10.0.0.1 9999             # TCP Echo (端口 9999)
```

Wireshark 抓包：

```
sudo wireshark -i tap0 -k &
# 可以看到完整的 ARP→ICMP/TCP 交互
```

模块依赖关系 (从底向上)

osal → *mbuf* → *timer* → *netif* → *driver* → *tap* → *ll_dispatch* → *arp* → *ip*

ip → (*icmp* / *route* / *udp* / *tcp*) → *socket* → *main*

event_bus: 独立, 任意模块可使用